

JavaScript Complete Reference Handbook

Comprehensive Core Fundamentals, Language Mechanics, Standard APIs & Advanced Concepts

JavaScript is a lightweight, single-threaded, and interpreted or just-in-time compiled programming language primarily known as the scripting language for web pages [cite: 1]. To master JavaScript, developers must understand its core building blocks, behavioral quirks, standard objects, and asynchronous execution loop [cite: 2, 3].

Part 1: Core Fundamentals & Language Mechanics

1. Variables, Scope & Immutability

- **Scope Types:** Global Scope (accessible everywhere), Function Scope (variables in `var` restricted to function bodies), and Block Scope (variables declared with `let` and `const` accessible only inside `{ }`) [cite: 2, 4].
- **var vs let vs const:** `var` is function-scoped and allows re-declaration/hoisting; `let` and `const` are block-scoped [cite: 4, 5]. `const` prevents reassignment but allows object property mutation [cite: 2, 4].
- **Accessing across files:** Use ES6 Modules (`export` / `import`) or attach properties to global objects (`window` in browser / `global` in Node.js).

2. Execution Context, Hoisting & Temporal Dead Zone (TDZ)

- **Hoisting:** Declarations are moved to top of scope during compilation [cite: 2, 6]. Functions are fully hoisted; `var` initializes as `undefined`.
- **TDZ:** Period between entering block scope and variable declaration where accessing `let` / `const` throws a `ReferenceError` [cite: 2, 6].
- **Strict Mode:** Invoked with `"use strict";` to throw errors on undeclared variables or mutating read-only properties [cite: 2, 6].

3. Function Mechanics, Closures & Context (this)

- **Closures:** Functions retain access to their outer lexical environment even after the outer function finishes executing [cite: 2, 7].
- **this Context:** Dynamic based on call-site [cite: 2, 7]. Arrow functions inherit `this` lexically from parent scope [cite: 7].
- **Explicit Binding:** Change execution context using `call(thisArg, p1, p2)`, `apply(thisArg, [args])`, or `bind(thisArg)` [cite: 2, 7].
- **Pass/Call by Value vs Reference:** Primitives (numbers, strings) pass by value; Objects/Arrays pass by memory reference [cite: 8].

```
// Closure Example
function createCounter() {
  let count = 0;
  return () => ++count; // Remembers count lexically
}
const counter = createCounter();
console.log(counter()); // 1
```

Part 2: Control Flow, Data Structures & Standard Built-ins

1. Operators & Conditionals

- **Operators:** Arithmetic (`+`, `-`, `*`, `/`, `%`, `**`), Comparison (`==` vs `===` for strict equality), Logical (`&&`, `||`, `??` nullish coalescing), Bitwise (`&`, `|`, `^`), and Ternary (`cond ? a : b`).
- **Coercion:** Implicit type conversion (e.g., `"5" + 2 => "52"`, `"5" - 2 => 3`). Explicit conversion uses `Number()`, `String()`, or `Boolean()`.
- **Conditionals & Loops:** `if-else`, `switch`, `for`, `while`, `do-while`, `for...in` (iterates object keys), and `for...of` (iterates iterable values).

2. Strings & Numbers

- **String Operations:** Immutably created. Methods include `slice()`, `indexOf()`, `replace()`, `trim()`, `includes()`, and template literals (``string ${var}``).
- **Numbers & Math:** Double-precision 64-bit IEEE 754 floating-point numbers. Use `Math.abs()`, `Math.floor()`, `Math.ceil()`, `Math.random()`, and `BigInt` for arbitrary precision integers.

3. Objects & Arrays

- **Creation:** Objects created via Literals (`{}`), `Object.create()` , Constructors, or Classes.
- **Array Methods:** `push()` / `pop()` (end), `unshift()` / `shift()` (start), `splice()` , `slice()` , `indexOf()` .
- **Higher-Order Array Methods:** `map()` (transforms elements), `filter()` (selects subset), `reduce()` (accumulates value), `forEach()` (iterates without returning new array) [cite: 7, 9].

```
// Array High-Order Transformation
const nums = [1, 2, 3, 4, 5];
const evensSquared = nums.filter(n => n % 2 === 0).map(n => n ** 2); // [4, 16]
```

4. Modern Data Structures (Set, Map, WeakMap, WeakSet, ArrayBuffer, Atomics)

- **Set & Map:** `Set` contains unique values; `Map` stores key-value pairs where keys can be any type (unlike objects).
- **WeakMap & WeakSet:** Hold weak references to objects, enabling automatic garbage collection when objects lose main references.
- **ArrayBuffer & DataView:** Raw binary data buffer representation used for low-level memory operations, WebAssembly, and WebSockets.
- **Atomics:** Provides atomic operations (e.g., `add` , `and` , `compareExchange` , `store`) on `SharedArrayBuffer` memory across web workers.

Part 3: Advanced JavaScript & Asynchronous Mechanics

1. Asynchronous Execution, Event Loop & Promises

- **Event Loop:** Single-threaded runtime concurrency model consisting of Call Stack, Web APIs, Macrotask Queue (`setTimeout` , `setInterval`), and Microtask Queue (`Promise` callbacks, `queueMicrotask`) [cite: 2, 7]. Microtasks always run before the next macrotask [cite: 7].
- **Promises:** Represent eventual async operation state (Pending, Fulfilled, Rejected) [cite: 2, 7, 8]. Methods include `Promise.all()` , `Promise.race()` , `Promise.allSettled()` , and `Promise.any()` .
- **Async/Await:** Syntactic sugar over Promises enabling synchronous-style readable asynchronous code [cite: 2, 7].

```
// Async API Request Example
async function fetchUserData(url) {
  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(`HTTP error! status: ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error("Fetch failed:", err);
  }
}
```

2. OOP, Classes & Metaprogramming (Proxy, Reflect, Symbol)

- **Classes & Prototypes:** ES6 classes provide object-oriented syntax over prototypal inheritance [cite: 9, 10, 11]. Supports constructors, inheritance (`extends`), `super` , getters/setters, static methods, and private fields (`#field`).
- **Symbols:** Unique, immutable primitive identifiers often used for non-colliding object property keys or built-in iterators (`Symbol.iterator`).
- **Proxy & Reflect:** `Proxy` intercepts core object behavior (get, set, construct) via traps [cite: 9]. `Reflect` provides standard functional API methods for target objects.

3. Error Handling, HTTP & Client Storage

- **Error Handling:** Block-level `try...catch...finally` , throwing custom `Error` instances, handling unhandled promise rejections.
- **HTTP & APIs:** `fetch()` API, `XMLHttpRequest` (AJAX), and REST operations (GET, POST, PUT, PATCH, DELETE).
- **Storage Comparison:**

Storage Type	Capacity	Expiration	Server Transmission
LocalStorage	~5-10 MB	Persistent until explicitly deleted	No
SessionStorage	~5 MB	Cleared when tab/window closes	No
Cookies	~4 KB	Configurable expiry date	Sent with every HTTP request